

Worked Solutions and Engineering Defense

Volume 3 · Complete assignment answers, assessment keys, capstone walkthroughs, and a sample architecture decision. Read after making your own prediction.

Harness engineering · Multi-agent orchestration · LangGraph

All lessons and reference materials are available now. The course uses explicit contracts, worked examples, executable local labs, and evidence-based design reviews. Interactive models and computational notebooks are included in the companion package.

Contents

Module 1: worked solutions	3
Module 2: worked solutions	4
Module 3: worked solutions	5
Module 4: worked solutions	6
Module 5: worked solutions	7
Module 6: worked solutions	8
Module 7: worked solutions	9
Module 8: worked solutions	10
Module 9: worked solutions	11
Module 10: worked solutions	12
Module 11: worked solutions	13
Module 12: worked solutions	14
Capstone reference walkthroughs	15
Additional trace and design keys	16

Module 1: worked solutions

Solution 01: Separate claim from outcome

The final sentence is a claim. Success requires order 7 to be refunded and exactly one matching 2,400-cent ledger entry, plus the starter lab's declared final-stop condition. An empty ledger fails regardless of wording. A useful trace contains proposal, validation/authorization, effect observation, final claim, and independent verification. Without approval, the executing service rejects the mutation; a later claim cannot change that state. Read the rejection and verification events rather than treating the tool-call attempt as success. A stronger contract names tenant, resource, permissible amount, authority source, deadline, and terminal outcomes. The starter's order-level approval is intentionally simpler than the exact-intent approval introduced in Module 5. Do not transfer its simplified permission representation unchanged into the capstone.

Reference: `python labs/run_lab.py 1`.

Solution 02: Diagnose the first actionable boundary

The service record establishes that O committed, although the timed-out client may not know it yet. A retry is a new attempt of the same logical operation O. Inventing O2 can create a second effect. Reconcile O before any new mutation. Record task/run/attempt/operation IDs, deadline, tool category, whether an effect may have committed, and the observed authoritative result. In the starter, malformed and blocked tool attempts consume the attempt budget; this prevents repeated invalid proposals from evading the ceiling. Pending means work remains; verified means the outcome predicate holds; denied means an authority boundary blocked the action; cancelled means no further work is admitted; uncertain effect requires reconciliation. An exception is an observation, not a complete business status. The first repair is the effect/recovery contract, not wording the retry prompt more forcefully.

Reference: `python labs/run_lab.py 1`.

Assessment key

Enter pending/accepted, retain the asynchronous job or operation ID, and wait for an authoritative terminal result. A transport acknowledgment is not the outcome. Test delayed completion and deadline expiry. On exhaustion, the truthful status is exhausted or pending-reconciliation, never success; the final message must carry the job identifier and a way to check the outcome later. A claim of completion at budget exhaustion is the false-completion case from Lesson 1.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 2: worked solutions

Solution 03: Compile context under a hard budget

Available input capacity is 80. Select the mandatory policy and the order evidence for 55 units; 25 remain unused because the 40-unit notes do not fit as a whole. The secret is excluded by tenant scope before ranking and is not an in-scope omission. With window 49 and reserve 20, only 29 units remain, so the mandatory 30-unit policy cannot fit and compilation fails. The compiler must renegotiate budget or task, not silently truncate the rule. Priority is a relevance policy inside an authorized candidate set, never a way to override scope. Whole-record selection and divisible allocation are different algorithms; their results need not match. Neither unit count is a provider tokenizer measurement. For a model-backed system, measure actual serialized tokens including wrappers, schemas, and response headroom.

Reference: `python labs/run_lab.py 2`.

Solution 04: Design an honest tool contract

Accept an exact positive integer amount at the shape boundary; reject True even though Python bool subclasses int, reject fractional and negative values, and reject the extra authority field. The service separately checks tenant ownership, current refundable state, exact amount paid, policy, and a trusted matching approval. Argument errors may permit correction; authority denials do not authorize searching for a bypass. A temporary read failure may be retried within budget. A write timeout must state that commit is uncertain and require reconciliation with a stable operation identity. Pagination should return items plus a stable cursor or explicit terminal marker, distinguish empty success from error, and bound response size. Schema validity alone does not establish domain validity, and a model-generated approval field cannot replace an authority record.

Reference: `python labs/run_lab.py 2`.

Assessment key

The other-tenant record is outside the authorized candidate set. Mandatory policy cannot be silently truncated. Fail or renegotiate the budget/task, preserve scope, and measure outcomes after a valid compilation.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 3: worked solutions

Solution 05: Build a layered grader

True positives are 67 because 82 positives include 15 false positives. False negatives are 5 because 72 outcomes are correct. True negatives are 13, giving 100 total. Precision is $67/82$, about 81.7%; recall is $67/72$, about 93.1%. Outcome checks inspect authoritative state; process checks inspect required steps; safety checks inspect forbidden actions. A high process score cannot repair a wrong outcome. Calibration should include the four named cases plus a correct refusal and a superficially failed response whose underlying outcome is correct. Preserve an uncertain category when truth cannot be established. Split by underlying customer/scenario family before creating paraphrases, and document which examples were used to tune the grader. A judge calibrated on one mixture may behave differently after the model or task distribution changes.

Reference: `python labs/run_lab.py 3`.

Solution 06: Calculate repeatability and uncertainty

There are $C(10,3)=120$ three-attempt subsets. Only two failures exist, so every three-attempt subset has at least one success: $\text{pass}@3$ is 1. All-success subsets number $C(8,3)=56$, giving $\text{pass}^3=56/120$, about 0.4667. The independent plug-in expression $0.8^3=0.512$ answers a different estimation question. Lab 3 reports the same paired mean difference (3/16) under task and cluster bootstraps. The task interval includes zero while the four-family interval does not, because three family means coincide and four clusters permit only 256 distinct resamples. The narrower family interval is the weaker evidence, not a significant result. Four families provide very limited information about population variation, so the intervals are teaching demonstrations, not strong evidence of generalization. Total cost is 6 and verified successes are 2, giving 3 units per success; a failure still costs resources. With zero successes, report undefined cost per success alongside total spend and zero completions. Do not replace the undefined denominator with zero.

Reference: `python labs/run_lab.py 3`.

Assessment key

Template dependence reduces the independent unit count; resample or split at the family level. Repeated candidate selection makes this development evidence optimistic. Use a fresh family-separated evaluation and report the full search budget. With $n=4$, $c=3$, $k=2$ there are $C(4,2)=6$ attempt pairs. The single failure cannot fill a pair by itself, so every pair contains a success and $\text{pass}@2=1-C(1,2)/C(4,2)=1$. Only the three all-success pairs count for $\text{pass}^2=C(3,2)/C(4,2)=0.5$. $\text{pass}@2$ answers whether a working attempt exists somewhere in two tries; pass^2 answers whether every attempt is reliable. The plug-in values 0.9375 and 0.5625 answer a different, model-based question.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 4: worked solutions

Solution 07: Prove a reducer contract

For compatible keyed values, merge is associative, commutative, and idempotent: $A+B$ contains a and b , and adding the identical a record changes nothing. A conflicting value for a is rejected rather than resolved by scheduling order. These algebraic claims hold on the compatible-record domain; the function is intentionally partial when conflicts occur. List concatenation is associative but generally neither commutative nor idempotent. Last-writer-wins may be a deliberate policy for some data, but it erases evidence conflict here. The join verifies that every expected job ID has exactly one compatible evidence record and that the job set is nonempty. This establishes coverage of the declared jobs, not truth of every source. Use a separate substantive verifier for source accuracy.

Reference: `python labs/run_lab.py 4`.

Solution 08: Budget dynamic workers

The parallel system uses seven calls and has a 12-second critical path: two worker rounds plus synthesis. The serial six-call baseline takes 24 seconds under the same fixed-duration assumptions and uses six calls. The comparison changes both parallelism and synthesis work; it is scheduling arithmetic, not a quality measurement. Missing evidence causes `verified=False`. An empty job set also returns false in this contract to avoid vacuous success. A product that permits partial coverage should return explicit expected/completed/missing IDs, a partial status, and limitations; it should not reuse the full-success predicate. Real latency includes queueing, tools, unequal durations, retries, and synthesis variability. Reserve worker and integration budgets before dispatch and account for stragglers and cancellation.

Reference: `python labs/run_lab.py 4`.

Assessment key

Reject conflicting values or define an explicit domain resolution rule with provenance. Test order independence on compatible updates and conflict detection separately. Do not allow scheduling order to decide truth. For the routing case, both verify and review are scheduled: a Command adds a dynamic edge and does not suppress a static edge. Use one routing mechanism per node and add a test that asserts the set of visited nodes.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 5: worked solutions

Solution 09: Recover the commit window

The effect database contains one refund and a refunded order. The controller has not recorded completion. Retrying the same intent while approval remains valid returns the original receipt with `duplicate=True` and leaves one effect. A new identity is not the same retry; in this particular service the already-refunded order also blocks a second refund, but other APIs may not provide that extra protection. A checkpoint and an external service commit are separate transactions, leaving an uncertainty window. After approval expiry, inspect the existing operation through an authorized reconciliation read. If committed, verify and recover the existing result. If absent, obtain a valid current decision before a new effect. Do not manufacture approval or treat a timeout as proof of non-execution. Exactly-once business outcome requires explicit service semantics, not the name of a workflow framework.

Reference: `python labs/run_lab.py 5`.

Solution 10: Bind approval and resume safely

The unchanged action at 39 is eligible if all other state checks hold. At 40 the approval is expired because validity is strictly before expiry. The changed amount fails the course's full-refund policy and no longer matches the approved intent. A policy-v2 execution cannot reuse the v1 decision under the example's current-policy check. A forged resume reference reaches the service but fails authorization. LangGraph resumes an interrupt by restarting its node, so operations before interrupt must tolerate reexecution; do not place an unprotected side effect there. Lab 5 keeps review separate from execution and reopens the SQLite checkpoint after an injected failure. For upgrades, pin paused tasks to a compatible runtime or explicitly migrate state and control-flow expectations, then test a real paused task. Never default a missing approval or status to success.

Reference: `python labs/run_lab.py 5`.

Assessment key

Expiry says the old record cannot authorize a new action now. It does not erase a committed effect. Reconcile through an authorized read, verify the existing outcome, and restore controller continuity without a new mutation. A fresh key per attempt turns each retry into a new logical operation, so the service cannot deduplicate and a timeout followed by retry can produce two effects. The key must identify the logical business operation, reused across attempts and bound to the canonical payload.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 6: worked solutions

Solution 11: Choose a topology from dependencies

Independent sources can use bounded workers plus evidence integration, compared with a strong single worker. Shared account mutation favors an explicit sequential workflow with service-side invariants; parallel proposals do not make dependent writes independent. Patch candidates can be generated independently and selected by meaningful acceptance checks, compared with one candidate given a comparable budget. Conflicting source review needs provenance-aware resolution; voting by number of agents is not a truth oracle. Lab 6 estimates calls, message copies, and idealized critical path. It cannot estimate model accuracy, source quality, or real queueing. The verifier sits at the outcome boundary in every design. State who owns mutable artifacts and how late or conflicting results are handled. A justified architecture can use fewer agents than the alternative.

Reference: `python labs/run_lab.py 6`.

Solution 12: Defend a multi-agent result

The observed success gain is six percentage points, but the interval includes a small loss, so the sample does not establish a clear positive effect. The median latency improvement may matter to a 30-second deadline, yet median alone does not show deadline compliance; request tail latency and timeout rates. Overnight work may not justify the extra cost absent stronger quality evidence. The stronger coordinator and changed verifier confound attribution to coordination. To isolate selection, freeze a candidate pool and compare deterministic verification, majority selection, and a discussion selector over that same pool under declared selection budgets. Then test the chosen configuration on fresh tasks. Report the whole-system result honestly while avoiding the universal claim that multi-agent is better.

Reference: `python labs/run_lab.py 6`.

Assessment key

The whole configuration improved under its tested conditions. Hold model/tool/evaluator opportunity comparable, vary coordination deliberately, include strong single-agent and independent-candidate baselines, and report actual cost and latency.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 7: worked solutions

Solution 13: Reject stale and conflicting artifacts

Accept the first A, treat identical A as a duplicate, reject revision-3 B as stale, reject W3 as the wrong owner, and reject conflicting C as a conflicting completion. A cancelled pending task rejects late output. The accepted artifact is copied so later mutation of the caller's object cannot alter the stored record. Add a required input_dependency_hash to the task and returned artifact; compare it to the current expected dependency before acceptance. A matching hash identifies exact input content but does not establish source truth or authorization. Preserve task identity across attempts, record attempt IDs for diagnostics, and keep integration acceptance separate from substantive verification. The local object demonstrates rules but is not a transactional distributed coordinator.

Reference: `python labs/run_lab.py 7`.

Solution 14: Audit a protocol boundary

Method discovery returns a schema; tools/call validates the envelope, tool name, and arguments before the bound-tenant lookup. Unknown methods produce a method error. Malformed arguments and tenant overrides fail the contract. A cross-tenant resource is unavailable even with a syntactically valid identifier. The example omits initialization/version negotiation, transport lifecycle, notifications, authentication, and broader protocol features; it must not be advertised as conformant. Deployment requires the official versioned SDK/specification, supported capabilities, authenticated transport, resource authorization, cancellation, limits, observability, and conformance tests. Product acceptance additionally checks that the tool performs the right scoped task and returns honest outcome semantics. A successful protocol handshake cannot authorize a refund or prove a research conclusion.

Reference: `python labs/run_lab.py 7`.

Assessment key

No. Check current task revision, ownership, cancellation, and dependency identity before integration. Protocol validity does not override application lifecycle or authority. Preserve the rejected artifact as diagnostic evidence if permitted.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 8: worked solutions

Solution 15: Verify a coding change

The next page exists only when a positive count advances the offset strictly below total. At the exact end, there is no next page; with zero count, the cursor cannot advance and should terminate under this fixture's contract. A useful added case is `offset=0,count=10,total=5`, which also returns `None` because the first page exhausts the dataset. The baseline must fail an acceptance case and the candidate must pass the declared suite; preserve both records and the final source digest. After interruption, inspect the actual diff and base revision, then rerun relevant checks rather than blindly reapplying the patch. The runner uses temporary directories and timeouts for trusted bundled Python. It is not a security sandbox and must not automatically execute arbitrary model-generated code in a privileged environment.

Reference: `python labs/run_lab.py 8`.

Solution 16: Reason about memory lifecycle

The observation is unavailable before time 10, available at 20, and expired at 30. Tenant B receives nothing. A tombstone prevents reuse of the deleted identifier in the local store. Full coverage requires $\text{ceiling}(23/5)=5$ calls; a four-call cap fails preflight before any classifier call. Do not present a 20-record partial count as a complete 23-record result. The newer forum comment does not automatically overrule the authoritative policy. Preserve both sources and dates, inspect whether the policy is still effective, and seek authoritative clarification when necessary. The store demonstrates local scope and expiry, not semantic truth or deletion from every replica, cache, backup, and derived summary. A real memory policy needs write gates and provenance-preserving transformations as well as retrieval.

Reference: `python labs/run_lab.py 8`.

Assessment key

The old result is historical evidence for another artifact/environment. Inspect current state and rerun relevant acceptance checks. Preserve provenance; do not promote remembered text to current verification. Full coverage needs $\text{ceiling}(23/5)=5$ calls, so the 4-call budget cannot complete the declared task; reject or renegotiate before the first call rather than returning a 20-record partial count as a total. If partial results are permitted, the contract must say so and the output must report the uncovered records.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 9: worked solutions

Solution 17: Threat-model the product action

Trusted service configuration binds actor to tenant; resource lookup enforces ownership; business logic enforces the full amount and current policy; the approval store binds an authorized principal to the exact intent and expiry; the effect ledger deduplicates the stable operation and checks payload consistency. The valid case commits one effect and the seven invalid fixed cases commit none. A hash detects content differences but anyone can compute a hash. Authority comes from a trusted authenticated principal and protected approval record, neither of which is established by a worker's sentence. The model and worker may propose or report evidence; the service executes only within its own checks. Passing the finite suite supports those covered invariants, not universal prompt-injection resistance or production identity security.

Reference: `python labs/run_lab.py 9`.

Solution 18: Measure defense and usefulness

TP=32, FN=8, FP=12, TN=48; precision=32/44≈72.7%, recall=80%. Pair wrong-tenant instructions with a valid scoped request, forged approval text with a real approval reference, changed-amount text with the exact approved amount, stale policy with current policy, and a malicious workflow document with a legitimate explanatory document. Measure authorized task completion on benign cases and forbidden reads/effects on attack cases. Detector classification alone is neither metric: an unflagged proposal can still be blocked by the service, and a flagged benign request may safely complete. Reject-all achieves no useful completion and therefore fails the product objective. Keep adaptive attack discovery separate from untouched evaluation and state attacker access, knowledge, and query budget.

Reference: `python labs/run_lab.py 9`.

Assessment key

Measure authorized completion on benign tasks, including legitimate external workflow guidance. Zero violations with zero usefulness fails the product contract. Evaluate end-to-end effects beyond detector labels.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 10: worked solutions

Solution 19: Fence a stale worker and reserve spend

A holds epoch 1 and B holds epoch 2. A's result is stale and rejected; B's current unexpired result is accepted. External effects still need their own idempotency or fencing because rejecting a queue completion cannot undo a remote write. The first 60-unit reservation succeeds, the 50-unit request fails because only 40 are available, and settlement records 35 spent with zero reserved, leaving 65 available. This example assumes one serialized budget owner; concurrent production settlement needs atomic reservation identities. One worker gives latencies 10,20,30,40; two give 10,10,20,20 under fixed service time and no other delays. These are deterministic scheduling outputs, not p95 estimates from a production distribution.

Reference: `python labs/run_lab.py 10`.

Solution 20: Recover an incompatible deployment

For A, reconcile the committed operation, verify its outcome, and repair controller continuity without a new effect. For B, resume through a compatible runtime or explicit migration with the same intent and valid decision. For C, retain the unexecuted state and obtain a valid current approval before execution. For D, reject automatic migration and route to a supported recovery path; do not invent defaults that grant authority. The teaching v1 migration defaults missing status to pending, while version 99 fails. A release gate should create an actual paused workflow under the old version and test resume/cancellation under the candidate deployment, including the effect window. Rolling back code does not undo external effects or incompatible memory writes; those need separate reconciliation and data repair.

Reference: `python labs/run_lab.py 10`.

Assessment key

Queue fencing rejects stale result acceptance but cannot undo the external write. The effect service must enforce idempotency, fencing, or another valid concurrency contract. Reconcile the external state. The budget case is a check-then-spend race: both reads precede either spend, so the budget is oversubscribed to 120. Reserve atomically before dispatch, settle actual usage afterward, and give each reservation an identity so one worker cannot settle another's allocation.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 11: worked solutions

Solution 21: Select a candidate without leaking the holdout

Keyword matching treats “not urgent” as urgent. The negation policy fixes the bundled contrast; the confirmation policy can miss urgent requests without “confirmed.” The supplied development set selects negation. The tiny holdout repeats lexical structures with different suffixes, so its result demonstrates selection mechanics rather than transfer to independent task families. Distinct IDs prevent literal overlap but not template leakage. A real search must split underlying families, freeze the objective and constraints, log candidate lineage and calls, and reserve a final evaluation population. Allocate the 30-call budget explicitly between proposals and candidate tests, count failures, and stop when exhausted. Reflection proposes hypotheses; independent evaluation decides promotion. This finite fixed-menu lab is not a GEPA or ACE reproduction.

Reference: `python labs/run_lab.py 11`.

Solution 22: Allocate compute by utility

The routes have expected utilities 0.60 and 0.50, so the less accurate route wins under this specific scalar objective. Different failure costs can reverse the choice. The verifier avoids expected loss $0.04 \times 10 = 0.40$, leaving 0.30 net value before latency and other effects. These probabilities require measurement; invented estimates cannot justify a production decision. The worker-plus-synthesis path needs 22 seconds and misses the remaining deadline. Perform the mandatory five-second check and return the appropriate verified or incomplete status. Hard authority and outcome checks are constraints, not optional utility trades. The seeded epsilon-greedy demo illustrates exploration and noisy value estimates in a stationary two-action model; it does not train an LLM or validate online learning on real user traffic.

Reference: `python labs/run_lab.py 11`.

Assessment key

The test has become development data. The result describes fitting to that set, not untouched generalization. Preserve the exposure history and use a new independent evaluation population before promotion.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Module 12: worked solutions

Solution 23: Assemble the evidence portfolio

The product trace should show one effect immediately after the crash, a duplicate receipt on retry, a restored checkpoint on the next call, and a true final-state verifier. The rejected trace must include the enforcing error and unchanged ledger. The coding negative control remains broken, while the reference patch moves from a failed baseline to passed acceptance checks and records the exact artifact digest. Record Python and dependency pins, fixture versions, commands, and test output. The optional live text adapter can collect one bounded proposal for manual inspection; do not feed it into the trusted local code runner automatically. A real coding experiment needs a security sandbox and independent task set. The portfolio must separate deterministic reference results from unmeasured live-model quality, scale, and production claims.

Reference: `python labs/run_lab.py 12`.

Solution 24: Defend transfer to an unfamiliar system

The first incident needs explicit asynchronous job state and outcome observation, with a pending terminal contract and a timeout test. The second needs evidence provenance, deduplication, and a selector comparison over a fixed candidate pool; parallelism can be retained if it earns its latency cost. The third needs a gated memory-update path, candidate lineage, rollback, and a fresh evaluation population; deleting one phrase is insufficient. In the new domain, define a real outcome oracle rather than copying refund-specific state fields. A good removal condition is measurable: the worker adds no independently verified coverage under equal budget, or the memory rule regresses fresh task families. The final defense should show what evidence supports the system and what could falsify its advantage. Clear limitations and justified simplification are signs of competence.

Reference: `python labs/run_lab.py 12`.

Assessment key

You can claim the tested fixture behaviors, state invariants, restart mechanics, and known reference-patch acceptance under pinned dependencies. Live-model capability, adversarial generalization, production scale, protocol conformance, and sandbox security require separate evidence.

Full credit requires the correct boundary, an enforceable repair, a meaningful test, and clear limits.

Capstone reference walkthroughs

Capstone A reference walkthrough

Run `python labs/run_lab.py 12 --output my-capstone` and inspect the product section. The first execution commits one effect then raises a simulated crash. The second execution returns the same receipt as a duplicate and writes controller completion. The third restores that checkpoint. The final verifier checks one matching effect and refunded state. Lab 5 adds an actual LangGraph interrupt and disk-backed checkpoint reopening. Lab 9 records zero effects for invalid fixed cases. The implementation is in `product.py` and `graphs.py`; the product graph uses a supplied deterministic clock, so a deployed adapter must provide a trusted current clock on each execution. A graph restart is not a distributed transaction. If approval expires after commit, authorized reconciliation establishes the existing result; if no effect exists, a fresh valid decision is needed.

A strong report separates these observations from live-model robustness. The actor map is trusted local configuration, SQLite is a local simulated service, and UUID receipt values vary across runs. Report stable invariant results rather than requiring identical receipt strings. The approval reference is not a user-controlled boolean. Reusing an operation identity with changed intent is a conflict, while an identical delivery is deduplicated. A second operation against the already-refunded order is rejected by the domain state rule.

Capstone B reference walkthrough

The unchanged candidate fails all six seeded acceptance suites; the reference candidate passes all six after the broken baseline fails. Inspect `coding.py` for the fixtures and `output/reference-run/artifacts/12/coding/` for saved source, checks, and reports. For pagination, strict end-boundary and positive progress prevent an extra page or nonadvancing loop. For money, exact integer typing rejects booleans. For tenant access, a public flag does not bypass tenant ownership in this task contract. For status, verification determines completion rather than a claim string.

These positive and negative controls validate the miniature acceptance environments. They do not estimate an LLM's coding performance. A useful extension introduces an independent defect and acceptance oracle, freezes the task before candidate generation, and records all attempts and costs. Inspect proposed live-model text manually or use a genuine constrained execution service; do not label the local subprocess runner a sandbox. The optional adapter has a fake-server contract test and has not been tested against a paid provider in this course build.

Example engineering decision memo

For the deterministic refund contract, retain an explicit workflow with service-side approval, deduplication, and verification. Additional agents do not improve the fixed reference policy's outcome and add integration cost. For independent evidence gathering under a deadline, evaluate bounded workers because the dependency graph permits parallel progress. The scheduling model suggests a possible latency benefit, but a live paired experiment is required to measure accuracy and actual tails. This decision is conditional on task population, resource limits, and measured evidence. Revisit it when the model, tools, or workload changes.

Additional trace and design keys

Lesson 3: grouped evidence

Ordinary greedy selects A and then cannot fit B or C, spending 30 units on an unusable half-pair. Group-aware selection sees the 60-unit pair cannot fit and selects C for 45, preserving an independently useful result. The compiler needs explicit dependency/group membership, scoped identities, combined size, priority/value policy, and rules for mandatory groups. Do not infer dependencies from adjacent text alone. A counterexample is two actually independent 30-unit records incorrectly grouped into a 60-unit candidate under a 50-unit budget: grouping excludes both, while ordinary selection can retain one useful record. Group awareness fixes a declared dependency problem; it is not universally superior to every greedy policy.

Lesson 5: ten outcomes

Case	Defensible classification	Metric treatment
1	False completion; requested state absent	Failure in valid-task completion denominator
2	Pending acknowledgment, not yet terminal	Keep pending until deadline; do not label success early
3	Verified authorized outcome	Success in valid-task completion denominator
4	Duplicate effect violates the task invariant	Failure plus separate duplicate-effect violation
5	Correct denial of an invalid scoped request	Success for appropriate handling; separate from valid-refund completion
6	Business effect correct; communication failed	Outcome success and response-delivery failure reported separately; overall product contract may require both
7	Unnecessary refusal	Valid-task completion failure; utility loss
8	Late unauthorized continuation after cancellation	Deadline failure and cancellation/effect violation
9	Evaluation infrastructure failure; truth unknown	Report ungraded count and coverage; investigate or rerun under a declared policy, never silently drop it
10	Correct read-only confirmation if evidence is current	Success under the actual query contract; no new effect required

The rubric must be fixed before comparing systems. Case 6 can be overall failure if reliable response delivery is part of the task; retain the separate outcome dimension so the cause remains visible. Case 9 should not make a system look better by disappearing from the report. Case 10 shows why a preexisting state can satisfy a query but does not establish that a requested new mutation was performed. A dataset card should state task types, validity strata, initial state, deadlines, grading coverage, and the split unit.

Lesson 6: subset enumeration

The six subsets are $\{S1, S2\}$, $\{S1, F1\}$, $\{S1, F2\}$, $\{S2, F1\}$, $\{S2, F2\}$, and $\{F1, F2\}$. Five have at least one success, so $\text{pass}@2=5/6$. Only the first has all successes, so $\text{pass}^2=1/6$. Plugging $p=0.5$ into an independent replacement model yields 0.75 and 0.25. The finite-sample estimator samples distinct recorded attempts without replacement; the plug-in model assumes independent new trials at an estimated fixed probability. State which question you are answering.

Lesson 7: combined routing

Both `static` and `dynamic` execute in the tested pinned runtime. The returned command schedules its destination; the explicit static edge also schedules work. The reference test compares the set of visited names, not an arbitrary sibling completion order. Remove the static edge if the dynamic command is intended to be the only route. This is a concrete example of why an intuitive graph sketch is insufficient without execution semantics.

Lesson 8: join policies

For full evidence coverage, A and C alone are incomplete because B is missing. Either fail the full-success predicate or return an explicitly permitted partial status with expected/completed/missing IDs and limitations. For the mutation task, reading version 7 is a dependency of the calculated update, and the commit must compare the current resource version with 7. If another writer changed it, reject/recompute under the current state rather than merging stale mutations. A fan-in barrier cannot make a stale precondition valid. The compiled-child example in Lab 4 projects only evidence back to the parent and keeps its private scratch field outside the parent result; this is an interface projection, not a proof that every streaming mode is redacted.

Lesson 10: approval transitions

State	Appropriate control
No approval	Pause or deny according to the product contract; no effect
Matching current approval	Execute only after current resource and policy checks
Revoked approval	Deny new execution; inspect existing effects through an authorized read if needed
Expired approval	Deny new execution; obtain a current decision if work remains
Changed intent	Revalidate and obtain a decision bound to the changed action
Duplicate committed delivery, valid approval	Return the existing receipt under service deduplication semantics
Committed effect, expired approval	Reconcile and verify the existing effect; do not invent a new grant
Unknown state version	Reject automatic resume; use supported migration or versioned/manual recovery

The table separates authority for a new action from knowledge of a past effect. The local service's simulated clock is supplied by the driver; a deployed service must own a trusted current clock rather than accept a model-selected or stale timestamp.

Lesson 12: architecture memos

Transactional memo: choose an explicit workflow for the fixed refund contract because the steps have a shared-state dependency and the service can enforce exact invariants. Compare a single proposal model with any proposed specialist under the same task/evaluator/budget. Additional agents must demonstrate useful capability beyond the deterministic reference, while approval and effect checks remain in the service. Reverse the decision if a measured specialist materially improves valid-task understanding without violating resource or authority constraints.

Evidence memo: test bounded workers when source inspections are independent and a deadline makes serial execution unsuitable. Reserve synthesis budget, preserve source identity, reject missing/conflicting output, and compare a strong single worker and independent-candidate baseline. The schedule suggests possible latency reduction under equal-duration assumptions; measure actual task success, tail latency, total spend, and source coverage in a paired live experiment. Reverse the decision if workers share redundant evidence, lose authoritative dissent, or fail to meet the deadline after integration overhead.